

Previsão de churn da rede Ford no momento da compra para geração de leads de pós-venda

Integrantes:

Lucca Borges — RM 554608
Ruan Melo — RM 557599
Rodrigo Jimenez — RM 558148
João Victor Franco — RM 556790
Bruno Leão — RM 555563

0,301

PR-AUC no teste
(aleatório = 0,142)

0,738

ROC-AUC no teste
(aleatório = 0,500)

43,7%

dos churners estão nos
20% de maior risco

2,19×

mais churners que uma
seleção aleatória (lift@20%)

1. Compreensão do problema

Contexto. O Desafio 02 da Ford busca aumentar o **VIN Share**, a porcentagem de veículos Ford que fazem manutenção na rede oficial. Cada cliente que migra para uma oficina independente reduz a receita de pós-venda e o vínculo com a marca. Este trabalho responde à frente "*Geração de Leads e Modelagem Preditiva*": identificar veículos em risco de sair da rede e gerar leads proativos para as concessionárias.

Objetivo da solução. No **momento da venda**, estimar para cada cliente a probabilidade de **churn da rede em 24 meses** (churn_rede_24m). Assim a concessionária concentra ações de retenção (plano de manutenção, agendamento da 1ª revisão, contato do consultor) nos clientes de maior risco.

Tipo de problema. **Classificação binária supervisionada** com saída probabilística. As classes são **desbalanceadas**: 14,2% de churn. Por isso usamos divisão e validação **estratificadas**, a **PR-AUC** (*Average Precision*) como métrica principal e métricas de negócio no top 20% de risco. A acurácia não serve aqui: "ninguém sai" já acertaria 85,8%.

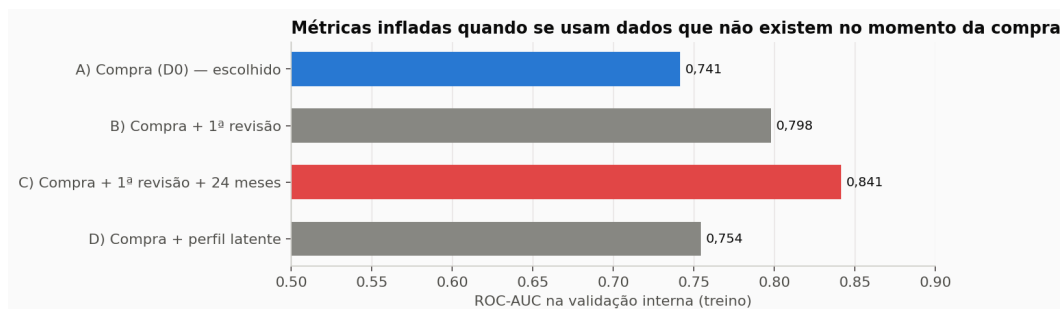
Dados. Foram fornecidos dois arquivos com os **mesmos 500 mil clientes**. O

ford_clientes_historico_completo.csv (37 colunas) é a **base de modelagem**, a única que tem o alvo. O ford_clientes_operacional_compra.csv é um **recorte exato** dele: só as 24 colunas conhecidas na venda, **sem o alvo**. Por isso não serve para treinar, mas define (i) quais variáveis o modelo pode usar e (ii) o **layout de entrada do modelo em produção**. Cada linha representa uma compra de veículo, **hipótese adotada neste protótipo**. Em produção, cada registro de compra seria associado ao VIN correspondente, e o churn da rede (veículo que deixa de fazer manutenção na rede) é o que reduz o VIN Share. Os 20% de teste foram separados **logo após a leitura**: exploração e decisões de desenvolvimento usam só o treino.

Principais achados da análise exploratória (no treino): churn de 7,2% para quem mora a ~3 km da concessionária contra 34,2% a ~56 km; 3,6% com plano de manutenção contra 16,5% sem; 21,6% entre os clientes menos organizados contra 4,1% entre os mais organizados; interior 17,2% contra capital 11,5%. Nenhuma variável isolada passa de $|\rho| = 0,19$ com o alvo e há relações não lineares, o que favorece modelos multivariados não lineares.

2. Preparação dos dados

Variáveis e vazamento de dados. O histórico tem colunas registradas **depois** da venda (1ª revisão) e na **mesma janela do alvo** (revisões, gasto, satisfação e share em 24 meses), além de um perfil_latente que não existe no mundo real. Tudo isso foi excluído. O experimento abaixo, feito só com dados de treino, mostra que essas colunas inflariam a PR-AUC de 0,306 para 0,458, com um modelo impossível de usar no momento da venda. Neste experimento, acrescentar o perfil latente elevou a PR-AUC de 0,306 para 0,318; o resultado não estabelece um teto de desempenho. Também saíram cliente_id (identificador) e categoria_veiculo (função exata de modelo_veiculo). Três variáveis criadas por conhecimento de negócio (valor financiado, valor/renda, revisões esperadas pelo km) foram **testadas com validação cruzada e descartadas**, porque não trouxeram ganho (-0,0002 de PR-AUC). O modelo usa **22 variáveis da compra**.



ROC-AUC na validação interna do treino por conjunto de variáveis: em vermelho, o modelo que usaria dados que ainda não existem na venda.

Tratamentos aplicados

- **Ausentes** (12 colunas no treino, todas < 2,5%; churn com o campo ausente próximo ao do campo presente, diferença máxima de 1,3 p.p.): imputação por **mediana** (numéricas) e **moda** (binárias/categóricas), justificada pela baixa proporção. No teste qui², só teve_ford_antes tem $p < 0,05$ (nominal, sem correção para 12 testes).
- **Inconsistências**: financiamentos com prazo 0 e prestação 0 (267 no treino) viram ausentes e são imputados; a regra é o 1º passo do pipeline. Também foram verificados: à vista × entrada/prazo, faixas válidas, modelo × categoria, piso de R\$ 55 mil e duplicatas (nenhuma).
- **Outliers** (caudas longas no treino: renda até R\$ 179 mil, distância até 364 km, parcela até 2,5× a renda): **winsorização** nos percentis 0,5–99,5, sem descartar clientes.
- **Transformação**: $\log(1+x)$ nas variáveis assimétricas, **padronização** (z-score) e **one-hot encoding** nas categóricas.
- **Divisão**: treino 80% (400.000) / teste 20% (100.000), **estratificada**. Todo parâmetro aprendido (quantis, medianas, médias) fica dentro de um Pipeline ajustado **só no treino**, inclusive em cada fold da validação cruzada.

3. Desenvolvimento dos modelos

Algoritmo	Por que foi escolhido	Configuração
Dummy (prevalência)	piso de referência: um modelo que não aprende nada	—
Regressão Logística	<i>baseline</i> linear, interpretável e com probabilidades calibradas	L2, C = 1
Árvore de Decisão	regras explicáveis; capta não linearidades	max_depth 8, min_samples_leaf 200
Random Forest	<i>bagging</i> : reduz a variância da árvore; robusta a outliers	200 árvores, profundidade 14
XGBoost	<i>boosting</i> : estado da arte em dados tabulares, com regularização	400 árvores, lr 0,05, profundidade 5
Rede Neural (MLP)	rede <i>feedforward</i> para relações não lineares complexas	64→32 ReLU, Adam, early stopping

SVM com kernel e KNN foram descartados por custo computacional com 400 mil linhas de treino (treino entre $O(n^2)$ e $O(n^3)$ no SVM; comparação com todo o treino a cada predição no KNN).

Treinamento e configurações testadas (validação cruzada estratificada 5-fold e holdout interno, sempre no treino):

- **Pesos de classe** (class_weight, scale_pos_weight): não melhoraram a PR-AUC (0,2727 → 0,2678 na Regressão Logística) e quase dobraram o Brier (probabilidades distorcidas). Os modelos ficaram sem pesos.
- **Profundidade da árvore** (2 a sem limite): melhor validação com profundidade 8 (0,2865). Sem limite, a árvore decora o treino (PR-AUC 1,0) e cai para 0,158 na validação.
- **Arquitetura da MLP**: 1 camada 0,2856, **2 camadas 0,2939**, 3 camadas 0,2854.
- **Ajuste de hiperparâmetros do XGBoost** (melhor modelo na validação cruzada): RandomizedSearchCV com 30 configurações × 3 folds, otimizando PR-AUC. A configuração vencedora é bem mais regularizada (285 árvores, lr 0,020, profundidade 5, min_child_weight 51, reg_lambda 10,6, gamma 1,51). Ela mantém a PR-AUC (0,3047 contra 0,3046) e **reduz o gap treino–validação observado em 78%** (0,053 → 0,012).

4. Avaliação e comparação dos modelos

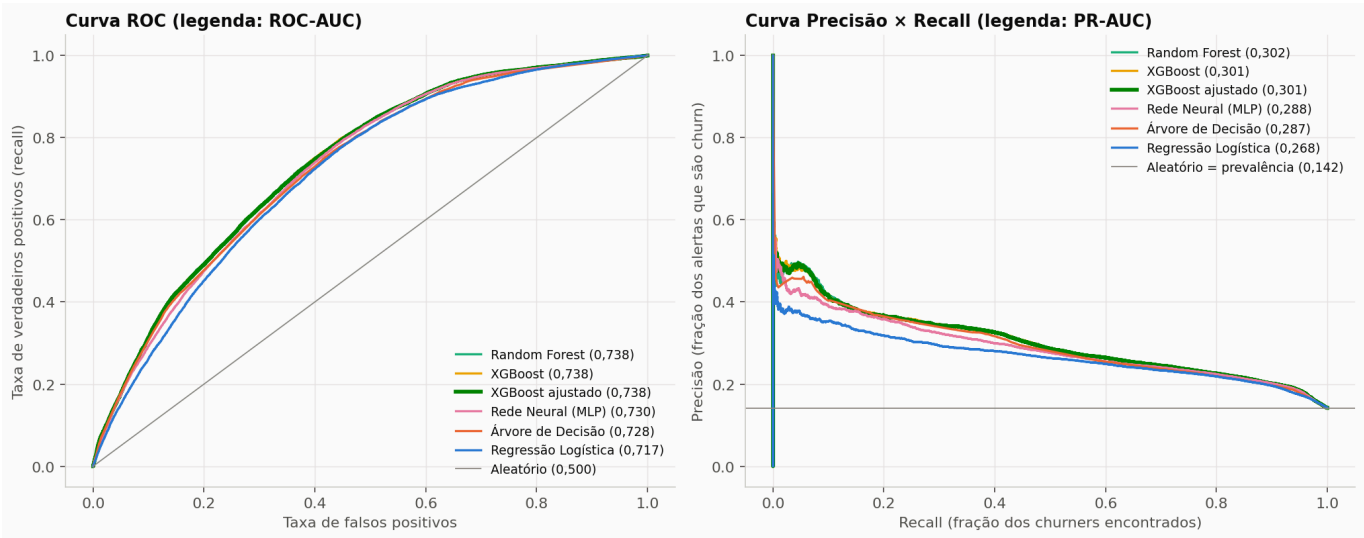
Validação cruzada estratificada 5-fold (treino, 400.000 clientes)

modelo	PR-AUC (média)	PR-AUC (dp)	ROC-AUC (média)	Brier (média)	gap treino–validação (PR-AUC)
XGBoost ajustado	0,3047	0,0028	0,7403	0,1104	0,0119
XGBoost	0,3046	0,0024	0,7401	0,1104	0,0533
Random Forest	0,3041	0,0029	0,7404	0,1105	0,0821
Árvore de Decisão	0,2908	0,0020	0,7318	0,1111	0,0108
Rede Neural (MLP)	0,2846	0,0053	0,7291	0,1119	0,0069
Regressão Logística	0,2724	0,0031	0,7209	0,1129	0,0003
Dummy (prevalência)	0,1419	0,0000	0,5000	0,1218	0,0000

Conjunto de teste (100.000 clientes fora do treino)

modelo	PR-AUC	ROC-AUC	Brier	Precision@20%	Recall@20%	Lift@20%
Random Forest	0,302	0,738	0,111	31,2%	43,9%	2,20×
XGBoost	0,301	0,738	0,111	31,1%	43,9%	2,19×
XGBoost ajustado	0,301	0,738	0,111	31,0%	43,7%	2,19×
Rede Neural (MLP)	0,288	0,730	0,112	29,7%	41,8%	2,09×
Árvore de Decisão	0,287	0,728	0,112	30,4%	42,8%	2,14×
Regressão Logística	0,268	0,717	0,113	28,1%	39,6%	1,98×
Dummy (prevalência)	0,142	0,500	0,122	14,1%	19,9%	1,00×

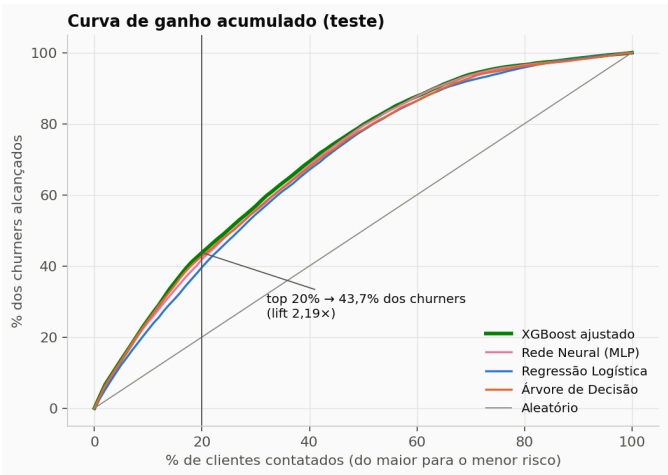
Métricas: PR-AUC = área sob a curva precisão × recall (acaso = 0,142); ROC-AUC = capacidade de ordenar churn acima de não churn (acaso = 0,5); Brier = erro quadrático das probabilidades (menor é melhor); Precision/Recall/Lift@20% = resultado se a concessionária contatar os 20% de clientes com maior score. Linha destacada = modelo selecionado.



Curvas ROC e Precisão × Recall de todos os modelos no teste.

Análise do desempenho

- **Todos os modelos aprendem padrões nos dados sintéticos** (PR-AUC 0,27–0,30 contra 0,142 do acaso). Os *ensembles* de árvores lideram. A Regressão Logística fica atrás, o que é coerente com as relações não lineares observadas na EDA.
- No teste, os três *ensembles* ficaram **muito próximos**. O intervalo de 95% (bootstrap) da diferença de PR-AUC entre o XGBoost ajustado e a Random Forest é [–0,0023; 0,0005] e contra o XGBoost padrão é [–0,0023; 0,0010]. Os dois incluem zero: nenhuma diferença foi detectada. O teste fica próximo da validação cruzada (diferenças ≤ 0,005), sem sinal de sobreajuste nas escolhas.
- **Modelo selecionado no teste:** no limiar F1-ótimo (0,179, definido no treino), *recall* 57,6% e *precision* 26,8%. Por faixa de risco (cortes do treino), a **faixa Alta** (10% dos clientes) tem churn de **35,4%** (2,50× a média). Alta + Média (30% dos clientes) reúnem **57% dos churners**. As probabilidades são bem **calibradas** e a ROC-AUC fica entre 0,70 e 0,76 em todas as regiões, categorias e canais.
- **Sanidade:** sem nunca ver o perfil latente, o modelo dá score médio de 23,5% aos clientes "abandono" e 3,7% aos "fiéis", na mesma ordem do churn real.



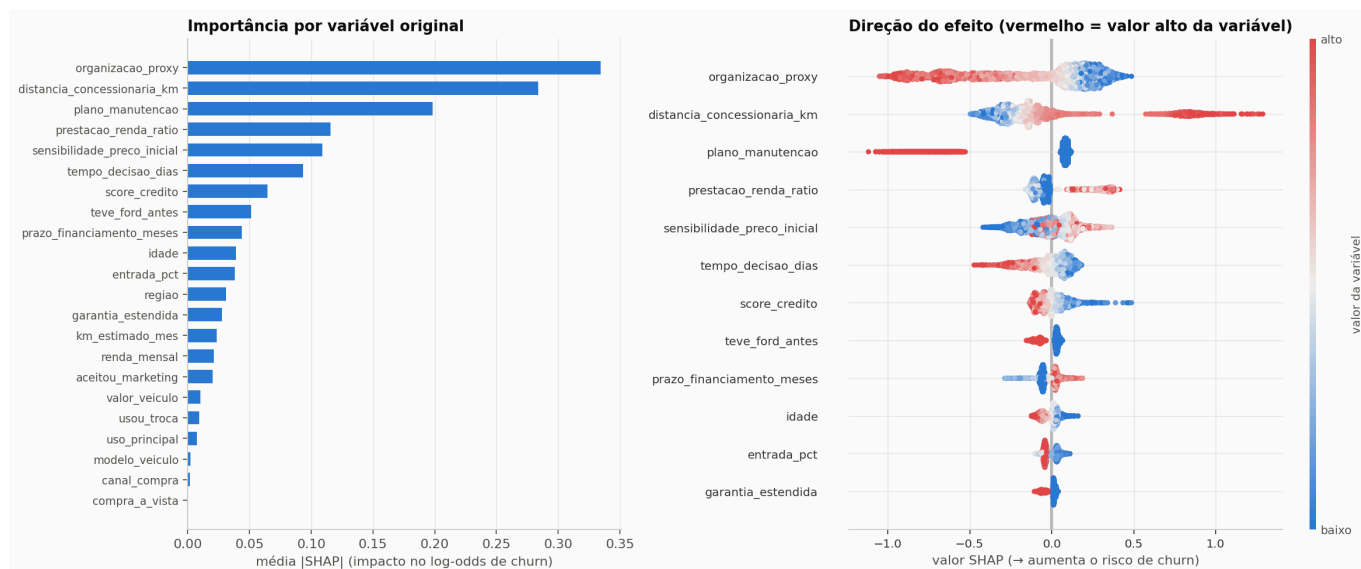
Curva de ganho no teste: contatando os 20% de maior risco, a concessionária alcança 43,7% dos churners.

5. Conclusão

Modelo selecionado: XGBoost ajustado (pipeline completo com as 22 variáveis da compra). **Justificativa:**

Critério	XGBoost ajustado	XGBoost padrão	Random Forest
PR-AUC validação cruzada (métrica principal)	0,3047	0,3046	0,3041
Gap treino–validação (overfitting)	0,012	0,053	0,082
PR-AUC teste (sem diferença detectada)	0,3007	0,3014	0,3017
Tamanho do modelo / tempo para pontuar 100 mil clientes	0,7 MB / 0,18 s	1,1 MB / 0,20 s	55 MB / 0,35 s

O XGBoost ajustado teve a **maior PR-AUC média na validação cruzada**, a métrica principal. No teste não se detectou diferença para os outros dois. Como argumentos adicionais, ele tem o **menor gap treino–validação observado** e é o **mais leve para produção**. Suas probabilidades são calibradas e ele é explicável com SHAP. **Principais resultados:** PR-AUC 0,301 (2,1× o acaso) e ROC-AUC 0,738 no teste. Os **20% de maior risco concentram 43,7% dos churners** (lift 2,19×). Os fatores de risco são acionáveis: baixa organização, distância da concessionária, ausência de plano de manutenção, parcela alta em relação à renda e sensibilidade a preço.



Fatores de risco do modelo selecionado (SHAP): à esquerda, a importância de cada variável; à direita, a direção do efeito (vermelho = valor alto da variável; à direita do zero = aumenta o risco).

Uso na aplicação da Ford e forma de deploy

1. Venda registrada

DMS/CRM da concessionária envia os dados da compra (layout do arquivo operacional_compra)



2. API de scoring

REST + JWT · POST
/api/v1/churn/score
carrega o pipeline .joblib (0,7 MB)



3. App / dashboard da concessionária

lista de leads priorizada:
Alto: consultor + plano de manutenção ·
Médio: lembretes + agendamento online ·
Baixo: comunicação padrão

4. Monitoramento e re-treino: logs dos scores + desfechos em 24 meses → *drift* das variáveis (PSI), calibração, PR-AUC real → re-treino periódico e versionamento do modelo.

O Pipeline completo (correção de inconsistências + pré-processamento + modelo) é salvo num único arquivo, junto com os cortes das faixas, e o código de pré-processamento fica no módulo `ford_churn_pipeline.py`. Assim a produção aplica exatamente as mesmas transformações do treino. A API (por exemplo, FastAPI em contêiner, protegida por JWT como o resto do projeto) pontua cada venda em **tempo real** ou em **lote noturno**: são 0,18 s para 100 mil clientes. O notebook simula esse fluxo: o **artefato de inferência é carregado e validado em processo Python separado**, nas versões documentadas, a partir do CSV operacional cru (2.353 clientes do teste, incluindo 63 financiamentos com prazo 0 e 532 registros com ausentes). As probabilidades coincidem com as avaliadas no teste (maior diferença absoluta: 1,1e-16).

Limitações e melhorias futuras

- Limitações:** os dados são sintéticos (é preciso revalidar com dados reais). A base completa foi consultada na exploração preliminar; embora a execução apresentada restrinja a exploração ao treino, essa exposição anterior limita a independência da avaliação. O modelo identifica o risco; o *lift* de identificação não é *lift* de retenção, e o efeito das ações sobre o VIN Share precisa ser medido com teste A/B. Não há datas de venda, então não houve validação temporal. Variáveis como `organizacao_proxy` e `sensibilidade_preco_inicial` precisam de um processo padronizado de coleta nas concessionárias.
- Segundo modelo pós-1ª revisão:** os dados da 1ª revisão elevam a PR-AUC de 0,306 para 0,414 (experimento de vazamento). Um segundo score, recalculado quando o cliente faz ou perde a 1ª revisão, refina os leads.
- Veículos conectados** (odômetro real, alertas de manutenção), **modelagem de uplift com testes A/B** das ações, **limiar orientado a custo** (custo do contato × valor do cliente retido) e governança **LGPD** com auditoria de viés.

Detalhes completos, código, tabelas e gráficos: notebook `Ford_Desafio2_IA_ML.ipynb` (repositório no GitHub e link do Google Colab no README). Os resultados quantitativos apresentados são extraídos dos resultados salvos desta execução (`figuras/resumo_resultados.json`).